STAT 184 - Homework 3: Data Wrangling & Reshaping

Due Friday, June 5 at 11:59 PM on Canvas

Your Name Here

AI Disclosure

☐ No AI used
☐ Syntax / Debugging (e.g., finding a missing comma, fixing an error code)
☐ Conceptual explanation (e.g., asking how `pivot_wider()` works)
☐ Code Generation (e.g., asking AI to write a pipeline from scratch)

Briefly describe your use (1-2 sentences):

[Type your explanation here]

Setup

Run the chunk below to load the necessary packages. Make sure `gapminder` and `nycflights13` are installed before you start.

- Run `install.packages(c("gapminder", "nycflights13"))` in the **Console** once if you haven't already. (`nycflights13` you already installed for HW2.)

```
library(tidyverse)
library(gapminder)
library(nycflights13)
```

This homework is mostly about `gapminder`, a dataset of life expectancy, population, and GDP per capita for 142 countries, recorded every five years from 1952 to 2007. Take a look:

```r
glimpse(gapminder)
```

```
Rows: 1,704
Columns: 6
$ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
$ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ~
$ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

Throughout, you'll be turning this raw table into answers, and, just as importantly, **checking that your answers are real**.

Question 1 - The six verbs

This question walks the data verbs from this week. Everything chains with the pipe (`|>`), one verb per line.

Part (a): filter + arrange

In a **single pipeline**, find the **10 countries with the highest life expectancy in 2007**. Keep all columns, just the right rows in the right order. (Hint: `filter()` to 2007, `arrange()` by `lifeExp` descending, `head(10)`.)

```r
# Write your R code here
```

In one sentence: which continent dominates the top 10, and is anything about that list surprising?

[Write your one-sentence answer here.]

Part (b): mutate

`gdpPercap` is GDP *per person*. The country's **total** GDP is population times GDP per capita. Add a new column `gdp_total = pop * gdpPercap`, then `select()` down to `country`, `year`, `pop`, `gdpPercap`, and your new `gdp_total` column. Show the `head()`.

```
# Write your R code here
```

Part (c): group_by + summarise

For **2007 only**, build a summary with **one row per continent** containing: the number of countries (`n()`), the mean life expectancy, and the total population.

```
# Write your R code here
```

> Tip
>
> If you see a message about `summarise()` having "regrouped" the output, that's informational, not an error. Add `.groups = "drop"` inside `summarise()` to silence it.

Part (d): a full pipeline

In **one pipeline**, answer: *Among countries with a population over 50 million in 2007, which five have the highest GDP per capita?* You'll need to `filter()` on two conditions, `arrange()`, `select()` sensible columns, and take the `head()`.

```
# Write your R code here
```

In one sentence, what do you notice about that list?

[Write your one-sentence answer here.]

Part (e): spot the errors

The chunk below has **two** mistakes. Run it once to see it fail, then write a fixed version that returns the mean life expectancy per year. In one or two sentences, explain what the two errors were.

```r
# Original (broken):
gapminder |>
  filter(continent = "Asia") |>
  group_by(gapminder, year) |>
  summarise(mean_life = mean(lifeExp))
```

```r
# Your fixed version:
```

[Write your explanation of the two errors here.]

Question 2 - Reshaping

Reshaping changes the *shape* of a table without changing its values. This question goes both directions.

Part (a): pivot_wider to compare two years

Build a table with one row per country and **separate columns for life expectancy in 1952 and in 2007**, then compute the change.

1. `filter()` to the years 1952 and 2007.
2. `select()` `country`, `continent`, `year`, `lifeExp`.
3. `pivot_wider()` with `names_from = year`, `values_from = lifeExp`.
4. `mutate()` a `change` column, and `arrange()` so the biggest gain is on top. Show the `head(10)`.

> **Warning**
>
> After pivoting, your new columns are named `1952` and `2007`. Because those names **start with a digit**, you must wrap them in backticks to use them: `` `2007` - `1952` ``.

```r
# Write your R code here
```

In one or two sentences: which countries gained the most life expectancy over this period, and what part of the world are they in?

[Write your answer here.]

Part (b): pivot_longer to tidy a wide table

Here is a small **wide** table of (rounded) GDP per capita, with one column per year:

```
gdp_wide <- tribble(
  ~country,         ~y1952, ~y1977, ~y2007,
  "China",             400,    741,   4959,
  "India",             547,    813,   2452,
  "United States",   13990,  25524,  42952
)
gdp_wide
```

```
# A tibble: 3 x 4
  country       y1952 y1977 y2007
  <chr>         <dbl> <dbl> <dbl>
1 China           400   741  4959
2 India           547   813  2452
3 United States 13990 25524 42952
```

Before you write any code, predict: if you pivot the three `y####` columns into a long form, how many **rows** and how many **columns** will the result have?

Prediction: [… rows / … columns]

Now pivot it longer, gathering the year columns into `year` / `gdpPercap`. (Hint: `cols = !country`.) Then add a `mutate()` that turns `"y1952"` into the number `1952` using `parse_number()`.

```
# Write your R code here
```

Did the result match your prediction?

[Write your answer here.]

Part (c): choose the shape

For each task, say whether you'd want the data **wide** or **long**, and which pivot gets you there. One sentence each, no code.

1. Plot each country's life expectancy over time, one line per country.
2. Compute, for each country, the difference in population between 1952 and 2007 as a single new column.

3. Group by continent and year to get an average life expectancy for each combination.

[1: …] — [2: …] — [3: …]

Part (d): CHALLENGE (optional)

This wide table crams **two** variables (`lifeExp`, `pop`) and a year into each column name:

```
mixed_wide <- tribble(
  ~country,   ~lifeExp_1952, ~pop_1952, ~lifeExp_2007, ~pop_2007,
  "Brazil",          50.9,    56602560,          72.4,   190010647,
  "Japan",           63.0,    86459025,          82.6,   127467972
)
mixed_wide
```

```
# A tibble: 2 x 5
  country lifeExp_1952 pop_1952 lifeExp_2007  pop_2007
  <chr>          <dbl>    <dbl>        <dbl>     <dbl>
1 Brazil          50.9 56602560         72.4 190010647
2 Japan           63   86459025         82.6 127467972
```

Using a **single** `pivot_longer()`, reshape this so the result has columns `country`, `year`, `lifeExp`, and `pop`. (Hint: see R4DS §5.3.3-5.3.4, you'll need `names_to = c(".value", "year")` and `names_sep = "_"`.)

```
# Your code (optional):
```

Question 3 - Trust, but verify

A classmate asked an AI assistant this question:

> *"For flights leaving JFK in summer, which airline (`carrier`) had the worst average arrival delay?"*

The AI produced the code below. It does **not** run as written, and even once it runs, it has problems.

```r
# AI-generated answer (do not trust yet):
flights |>
  filter(origin = "JFK", month > 6 & month < 9) |>
  group_by(carrier) |>
  summarise(avg_delay = mean(arr_delay)) |>
  arrange(desc(avg_delay))
```

Part (a): make it run, then find the problems

First, fix whatever **syntax error** stops it from running. Then identify **at least two** further problems that are about *logic or trustworthiness*, not syntax. For each, state: **what's wrong**, **how you found it**, and **the fix**.

> Things worth checking
>
> - Does it define "summer" correctly? Which months did it actually keep?
> - `arr_delay` has missing values (cancelled flights). What does `mean()` do with those by default?
> - Is the "worst" carrier based on enough flights to believe? What's missing from the output that would tell you?

Problem 1 — what / how found / fix:

[Describe.]

Problem 2 — what / how found / fix:

[Describe.]

Problem 3 (if you found one):

[Describe.]

Part (b): the verified version

Write **your own** corrected pipeline that you can stand behind. It should define summer as June–August, handle the missing values, and include a count so a tiny carrier can't top the list by accident. Then keep only carriers with a reasonable number of flights.

```r
# Your verified pipeline:
```

In one or two sentences: which carrier is *actually* the worst out of JFK in summer, on how many flights, and how confident are you?

[Write your answer here.]

Question 4 - Average over what?

Newspapers love a sentence like *"the average life expectancy in Asia is 70 years."* But an average is meaningless until you say **average over what**. Averaging the 33 Asian *countries* treats tiny Bhutan the same as enormous China. Averaging over *people* is a different number entirely.

In survey and demographic work this is the difference between an **unweighted** and a **population-weighted** estimate, and the two can disagree a lot.

Part (a): the naive average

For **2007**, compute one row per continent with the **unweighted** mean of `lifeExp` (the "average country") and the number of countries.

```
# Write your R code here
```

Part (b): the population-weighted average

Now compute the **population-weighted** mean for each continent: the "average person's" life expectancy. The formula is

$$\text{weighted mean} = \frac{\sum(\text{lifeExp} \times \text{pop})}{\sum \text{pop}}$$

which you can write inside `summarise()` using only `sum()`: `sum(lifeExp * pop) / sum(pop)`.

```
# Write your R code here
```

Part (c): put them side by side

In a single pipeline, build a table for 2007 with **both** estimates per continent, the number of countries, and a `diff` column (`weighted_mean - naive_mean`). Arrange by `diff`.

```
# Write your R code here
```

In **2-3 sentences**: which continent shows the biggest gap between the two estimates, and *why*? (Think about which countries are huge and whether they're typical of their continent.)

[Write your answer here.]

Part (d): show it

Take your table from part (c), `pivot_longer()` the two mean columns into one `estimate_type` / `value` pair, and make a plot comparing the naive and weighted estimates across continents. (A grouped/colored point or column chart both work. Label your axes.)

```
# Write your R code here
```

In one sentence, what does the plot make visible that the raw numbers didn't?

[Write your answer here.]

Part (e): which would you report?

In **2-3 sentences**: a reporter asks you for "Asia's life expectancy" to put in a headline. Which of your two numbers do you give them, and why? Is there a case for reporting both? Connect your answer to the idea that *every average has a hidden "average over what."*

[Write your answer here.]

Code appendix

```r
library(tidyverse)
library(gapminder)
library(nycflights13)
glimpse(gapminder)
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Original (broken):
gapminder |>
  filter(continent = "Asia") |>
  group_by(gapminder, year) |>
  summarise(mean_life = mean(lifeExp))
# Your fixed version:
# Write your R code here
gdp_wide <- tribble(
  ~country,         ~y1952, ~y1977, ~y2007,
  "China",             400,    741,   4959,
  "India",             547,    813,   2452,
  "United States",   13990,  25524,  42952
)
gdp_wide
# Write your R code here
mixed_wide <- tribble(
  ~country,   ~lifeExp_1952, ~pop_1952, ~lifeExp_2007, ~pop_2007,
  "Brazil",          50.9,   56602560,         72.4,   190010647,
  "Japan",           63.0,   86459025,         82.6,   127467972
)
mixed_wide
# Your code (optional):
# AI-generated answer (do not trust yet):
flights |>
  filter(origin = "JFK", month > 6 & month < 9) |>
  group_by(carrier) |>
  summarise(avg_delay = mean(arr_delay)) |>
  arrange(desc(avg_delay))
# Your verified pipeline:
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
```